

Month 1 · Day 2

Binary Search Is a Boundary-Finding Algorithm

Two-hour guided coding module · GATE + interview practice · C++17

Central question: When sorted data changes from “too small” to “large enough,” how do we find the exact change point without an off-by-one error?

Session map

Time	Activity	Discovery
0–15 min	Exact search	Comparisons eliminate whole regions
15–40 min	Lower bound	Search can return a meaningful boundary without equality
40–55 min	Hand traces	Interval conventions control correctness
55–70 min	Upper bound and duplicates	A run is described by two boundaries
70–110 min	Three coding labs	Direct use, contract variation, transfer
110–120 min	Review and exit ticket	Explain the invariant without code

Learning objectives

By the end of today, you should be able to:

1. Implement exact binary search using a closed interval.
2. Implement lower and upper bounds using a half-open interval.
3. Explain why every interval update is safe and makes progress.
4. Handle empty arrays, missing targets, endpoints, and duplicate runs.
5. Use `n` as a valid insertion boundary without indexing `a[n]`.
6. Find the first and last occurrence in $O(\log n)$ time.
7. Detect infinite-loop and off-by-one bugs in binary-search code.

1. Motivation: a sorted student-ID index

Suppose a course system stores student IDs in nondecreasing order:

TEXT

[101, 104, 104, 104, 112, 119, 130]

The same data supports several different questions:

- Does 112 exist?
- Where should 110 be inserted?

- Where does 104 first appear?
- Where does the run of 104 end?
- How many records have ID 104?
- What is the first ID greater than or equal to 110?

The input is unchanged. The output contract changes.

Today's discovery: Binary search is not one memorized template. It is a method for locating a boundary in an ordered decision space.

Warm-up: exploit the order

For target 119, a linear scan might inspect six entries. If we inspect the middle value 104, we immediately know that every position to its left is also too small. One comparison eliminates an entire region.

Checkpoint 1

Using the ID array above, answer without code:

1. Where should 100 be inserted?
2. Where should 105 be inserted?
3. Where should 140 be inserted?
4. What are the first and last indices containing 104?
5. Which answers are valid positions but not valid indices of existing elements?

2. Exact search with a closed interval

Begin with the familiar question: return an index containing target, or -1 if it is absent.

Use the closed interval:

TEXT

[low, high]

Both endpoints are active candidates.

Exact-search invariant

If the target exists and has not yet been returned, at least one occurrence lies inside [low, high].

CPP

```
int exactSearch(const vector<int>& a, int target) {
    int low = 0;
    int high = static_cast<int>(a.size()) - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (a[mid] == target) return mid;
        if (a[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}
```

Why the updates are safe

If $a[mid] < \text{target}$, sorted order implies every index at or left of mid also contains a value below the target. The entire region $[low, mid]$ is impossible, so the next interval begins at $mid + 1$.

If $a[mid] > \text{target}$, the symmetric argument eliminates $[mid, high]$.

Why the loop terminates

Every unsuccessful iteration removes mid . The interval length strictly decreases. The empty closed interval is represented by $low > high$.

Overflow-safe midpoint

Prefer:

CPP

```
int mid = low + (high - low) / 2;
```

Mathematically it equals $(low + high) / 2$, but it avoids adding two potentially large positive indices.

Trace A: target present

TEXT

```
a = [1, 3, 5, 7, 9, 11, 13]
target = 11
```

low	high	mid	a[mid]	Decision
0	6	3	7	Too small; low = 4
4	6	5	11	Found

Trace B: target absent

TEXT

```
a = [1, 3, 5, 7, 9, 11, 13]
target = 8
```

low	high	mid	a[mid]	Decision
0	6	3	7	Too small; low = 4
4	6	5	11	Too large; high = 4
4	4	4	9	Too large; high = 3
4	3	—	—	Empty interval; absent

Checkpoint 2

Trace exact search on:

TEXT

```
[ ] with target 4
[5] with target 5
[5] with target 2
[1,3,5,7] with target 7
[1,3,5,7] with target 4
```

For each trace, state when the interval becomes empty.

3. The more general question: first valid position

Exact equality is sometimes the wrong question. Suppose the target is absent but we still need its insertion position.

For sorted array:

TEXT

```
[1, 3, 3, 3, 7, 9, 14]
```

the insertion position for 4 is index 4. We want the first index satisfying:

TEXT

```
a[index] >= target
```

This is called a lower bound.

Boolean-region view

For target 4, label each position by the predicate `a[i] >= 4`:

TEXT

```
values:    [1,    3,    3,    3,    7,    9,    14]
predicate: [false, false, false, false, true, true, true]
                        ^
                    first true position
```

Binary search finds the boundary between the false and true regions.

Half-open interval

Use:

TEXT

```
[low, high)
```

`low` is included; `high` is excluded. Begin with `low = 0` and `high = n`.

Allowing `high = n` matters because insertion after every existing value is legitimate.

Lower-bound invariant

- Every index strictly before `low` is known to contain a value `< target`.
- Every index at or after `high` is known to contain a value `>= target`.
- The first valid position remains in the boundary range `[low, high]`.

CPP

```
int lowerBound(const vector<int>& a, int target) {
    int low = 0;
    int high = static_cast<int>(a.size());

    while (low < high) {
        int mid = low + (high - low) / 2;
        if (a[mid] < target) {
            low = mid + 1;
        } else {
            high = mid;
        }
    }
    return low;
}
```

Why `high = mid`, not `mid - 1`?

When `a[mid] >= target`, `mid` itself may be the first valid position. We must keep it. In a half-open interval, assigning `high = mid` retains `mid` as the excluded boundary candidate while discarding everything strictly after it.

Why return `low`?

The loop stops only when `low == high`. At that moment:

- all earlier indices are invalid because their values are too small;
- the meeting point is the first valid position;
- if the meeting point equals `n`, no array value is large enough and insertion belongs at the end.

Detailed trace: lower bound of 4

TEXT

`a = [1, 3, 3, 3, 7, 9, 14]`

low	high	mid	a[mid]	Predicate	Update
0	7	3	3	false	low = 4
4	7	5	9	true	high = 5
4	5	4	7	true	high = 4
4	4	—	—	—	return 4

Boundary gallery

For `[1, 3, 3, 3, 7, 9, 14]`:

TEXT

```
lowerBound(0) = 0
lowerBound(1) = 0
lowerBound(3) = 1
lowerBound(4) = 4
lowerBound(14) = 6
lowerBound(20) = 7
```

Checkpoint 3

For each result, write the false region and true region explicitly. Why must the return value for 20 be allowed to equal the array length?

4. Upper bound: the other side of a duplicate run

An upper bound is the first index satisfying:

TEXT

`a[index] > target`

Only one comparison changes:

CPP

```
int upperBound(const vector<int>& a, int target) {
    int low = 0;
    int high = static_cast<int>(a.size());

    while (low < high) {
        int mid = low + (high - low) / 2;
        if (a[mid] <= target) {
            low = mid + 1;
        } else {
            high = mid;
        }
    }
    return low;
}
```

Lower versus upper bound

Function	First index satisfying	Values definitely left of boundary
Lower bound	$a[i] \geq \text{target}$	$< \text{target}$
Upper bound	$a[i] > \text{target}$	$\leq \text{target}$

For target 3 in [1, 3, 3, 3, 7, 9, 14]:

TEXT

```
lowerBound(3) = 1
upperBound(3) = 4
first index   = 1
last index    = 4 - 1 = 3
count         = 4 - 1 = 3
```

Exact search derived from lower bound

CPP

```
int exactSearchWithLowerBound(const vector<int>& a, int target) {
    int position = lowerBound(a, target);
    if (position < static_cast<int>(a.size()) && a[position] == target) {
        return position;
    }
    return -1;
}
```

The order of the condition matters: never evaluate $a[\text{position}]$ when $\text{position} == n$.

First and last occurrence

CPP

```
pair<int, int> firstAndLast(const vector<int>& a, int target) {
    int first = lowerBound(a, target);
    if (first == static_cast<int>(a.size()) || a[first] != target) {
        return {-1, -1};
    }
    int afterLast = upperBound(a, target);
    return {first, afterLast - 1};
}
```

Checkpoint 4

Give lower bound, upper bound, first index, last index, and count for targets 1, 3, 8, 14, and 20 in:

TEXT

```
[1, 3, 3, 3, 7, 9, 14]
```

5. Debugging binary search

Consider this lower-bound attempt:

CPP

```
while (low < high) {
    int mid = low + (high - low) / 2;
    if (a[mid] < target) {
        low = mid;        // bug
    } else {
        high = mid;
    }
}
```

If `high = low + 1`, then `mid == low`. Assigning `low = mid` changes nothing, so the loop can repeat forever.

Repair it with:

CPP

```
low = mid + 1;
```

Common failure modes

- Mixing a closed loop condition with half-open updates
- Discarding `mid` when it might be the first valid position
- Keeping `mid` on the branch where it is definitely invalid
- Indexing the array at the returned position without checking `position < n`
- Finding any duplicate instead of the requested first or last duplicate
- Finding one duplicate and scanning outward, losing logarithmic complexity
- Writing code before deciding what the active interval means

Debugging exercise

For each faulty fragment, give a smallest failing input:

CPP

```
// A
while (low < high) { ... high = mid - 1; }

// B
int mid = (low + high) / 2;

// C
int p = lowerBound(a, target);
if (a[p] == target) return p;
```

Explain whether the failure is incorrect output, overflow risk, out-of-bounds access, or nontermination.

6. Coding Lab A: exact search

LeetCode 704 — Binary Search

Official problem: <https://leetcode.com/problems/binary-search/>

Why this problem is here

This is the direct implementation test. The sorted input makes the pattern visible; the challenge is maintaining a precise interval and making progress on every branch.

Before coding

Write these three lines in a comment:

TEXT

```
Interval convention:  
Loop condition:  
Meaning of an empty interval:
```

Required trace cases

TEXT

```
[], target 4  
[5], target 5  
[5], target 2  
[1,3,5,7], target 7  
[1,3,5,7], target 4
```

Progressive hints

1. Use a closed interval `[low, high]`.
2. Continue while `low <= high`.
3. On an unsuccessful comparison, exclude `mid` from the next interval.
4. Return `-1` only after the interval becomes empty.

Post-attempt questions

- Which invariant did your code maintain?
- Why can neither update leave the interval unchanged?
- What would change if you used `[low, high)`?

Target: $O(\log n)$ time and $O(1)$ auxiliary space.

7. Coding Lab B: insertion boundary

LeetCode 35 — Search Insert Position

Official problem: <https://leetcode.com/problems/search-insert-position/description/>

Why this problem is here

Equality is no longer necessary. The answer is the first position at which the target could appear without breaking sorted order—the lower bound.

Do not use

- Append followed by sorting
- A second linear scan
- A separate case for every endpoint

Required trace cases

TEXT

```
[1,3,5,6], target 5 -> 2  
[1,3,5,6], target 2 -> 1  
[1,3,5,6], target 7 -> 4  
[1,3,5,6], target 0 -> 0
```

Progressive hints

1. Search $[0, n)$.
2. Ask only whether $a[mid]$ is too small.
3. If it is too small, discard it and everything before it.
4. Otherwise keep mid as a possible first valid position.
5. Return the meeting point.

Post-attempt questions

- Why can the return value equal n ?
- What does every index before the answer contain?
- Can this same helper find an existing target?

Target: $O(\log n)$ time and $O(1)$ auxiliary space.

8. Coding Lab C: duplicate boundaries

LeetCode 34 — Find First and Last Position of Element in Sorted Array

Official problem: <https://leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array/description/>

Why this problem is here

The target may occupy a whole run. Finding any occurrence is insufficient. The problem is solved by composing two boundary searches.

Tempting but inadequate approach

Binary-search for one copy, then scan left and right. In an array containing only the target, the scan is $\Theta(n)$, violating the required logarithmic runtime.

Design before coding

TEXT

```
first = first index with value >= target
afterLast = first index with value > target
```

If $first$ is outside the array or does not hold the target, the target is absent. Otherwise return:

TEXT

```
[first, afterLast - 1]
```

Required trace cases

TEXT

```
[5,7,7,8,8,10], target 8 -> [3,4]
[5,7,7,8,8,10], target 6 -> [-1,-1]
[], target 0 -> [-1,-1]
[2,2,2,2], target 2 -> [0,3]
[1,2,3], target 1 -> [0,0]
```

Progressive hints

1. Write lower bound as a helper.
2. Either write upper bound or compute the lower bound of the next-greater condition.
3. Check $first < n$ before reading $a[first]$.

4. Two logarithmic searches remain $O(\log n)$.

Post-attempt questions

- How do the two predicates differ?
 - How would you return only the count?
 - Why is scanning outward not acceptable?
-

9. Consolidation exercises

Exercise 1: count occurrences

Write:

C++

```
int countOccurrences(const vector<int>& a, int target);
```

Use two boundaries. Do not scan the duplicate run.

Exercise 2: first value strictly greater than x

Given a sorted array, return the index of the first value strictly greater than x , or n if none exists. State the exact predicate your search uses.

Exercise 3: predecessor

Return the index of the largest value strictly less than x , or -1 if none exists. Derive it from a boundary rather than writing a new loop.

Exercise 4: library comparison

For several arrays and targets, compare your functions with C++:

C++

```
std::lower_bound  
std::upper_bound
```

Use randomized small arrays and assert that the indices agree.

Exercise 5: explain the contract

For each request, choose exact search, lower bound, or upper bound:

- Find any copy of x .
 - Find where x can be inserted before existing equal values.
 - Find where x can be inserted after existing equal values.
 - Count copies of x .
 - Find the first value at least x .
 - Find the first value greater than x .
-

10. GATE and interview concept check

Mark true or false and justify:

1. Binary search requires an array sorted in increasing order and cannot work on any other decision space.
2. Two successive $O(\log n)$ searches are still $O(\log n)$.
3. Lower bound always returns an index of an existing array element.
4. If $a[mid] \geq \text{target}$ during lower-bound search, mid may still be the answer.
5. A closed interval is empty when $low == high$.
6. A half-open interval $[low, high)$ is empty when $low == high$.
7. Finding a target and scanning across duplicates always preserves logarithmic runtime.
8. Binary search is fundamentally justified by monotonic elimination, not by memorized syntax.

11. Exit ticket

Complete without notes:

1. State the exact-search invariant.
2. State the lower-bound invariant.
3. Explain why lower bound uses $< \text{target}$ while upper bound uses $\leq \text{target}$.
4. Explain why $high$ begins at n in half-open boundary search.
5. Give a smallest input on which $low = mid$ can cause nontermination.
6. Explain how two bounds produce first index, last index, and count.
7. Name one place outside arrays where a first-true boundary search could be useful.

Self-rating

Skill	1	2	3	4
I can trace a closed binary search	☐	☐	☐	☐
I can implement lower bound from its invariant	☐	☐	☐	☐
I can implement upper bound without guessing	☐	☐	☐	☐
I can handle missing targets and n safely	☐	☐	☐	☐
I can find duplicate boundaries in logarithmic time	☐	☐	☐	☐
I can explain why every update is safe	☐	☐	☐	☐

Solution and instructor appendix

Stop here until you have attempted the traces, coding labs, and consolidation exercises.

Checkpoint 1

Insertion positions are 0 for 100, 4 for 105, and 7 for 140. The run of 104 occupies indices 1 through 3. Position 7 is a valid insertion boundary but is not an existing array index.

Checkpoint 2

The empty array begins with `high = -1`, so the loop never starts. For `[5]`, target 5 is found at index 0; target 2 makes `high = -1`. For `[1, 3, 5, 7]`, target 7 is found at 3; target 4 eventually leaves `low = 2` and `high = 1`.

Checkpoint 3

For target 4, indices 0..3 form the false region and 4..6 the true region. For target 20, every existing index is false, so the first true boundary is the conceptual position immediately after the array: `n = 7`.

Checkpoint 4

For `[1, 3, 3, 3, 7, 9, 14]`:

Target	Lower	Upper	First/last	Count
1	0	1	0/0	1
3	1	4	1/3	3
8	5	5	absent	0
14	6	7	6/6	1
20	7	7	absent	0

Debugging exercise

Fragment A mixes interval conventions and can skip a valid boundary. Fragment B carries integer-overflow risk for large indices. Fragment C reads out of bounds whenever lower bound returns `n`, such as `[1, 3]` with target 5 or the empty array.

Consolidation guidance

Occurrence count is `upperBound(a, target) - lowerBound(a, target)`. The first value strictly greater than `x` is `upperBound(a, x)`. The predecessor index is `lowerBound(a, x) - 1`, which naturally produces `-1` when every value is at least `x`.

Concept-check answers

1. False: binary search applies to any monotone decision space.
2. True: constant multiples do not change the growth class.
3. False: it may return `n`.
4. True: earlier values may still be smaller.
5. False: `[low, low]` contains one candidate in the closed convention.
6. True.
7. False: a duplicate run can contain all `n` elements.
8. True.

Instructor observation checklist

Record:

- Which interval convention the learner chose
- First incorrect boundary update
- Whether progress was checked on both branches

- Whether n was treated safely
 - Whether duplicate range was solved with two searches
 - Whether the learner could state the invariant before coding
 - Whether time and auxiliary space were explained correctly
-

Looking ahead to Day 3

Tomorrow uses sorted order differently. Instead of halving an interval, two pointers will eliminate rows or columns of possible pairs. The same standard remains:

Every movement must discard only possibilities that can no longer be correct.